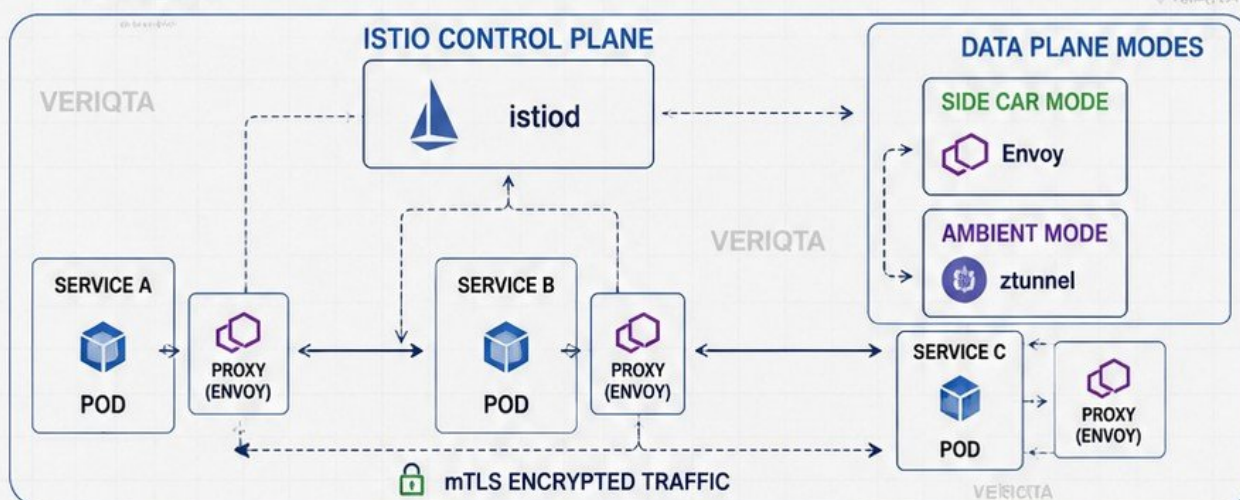
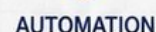
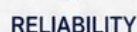
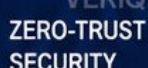




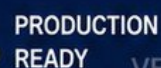
From Service Mesh Fundamentals to Production Traffic, Security, and Observability

VirtualService
DestinationRule

mTLS AuthorizationPolicy

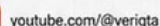


Metrics • Logs
Tracing • Kiali



Resilience Best Practices

VERIQTÀ SOCIAL



1. INTRODUCTION TO SERVICE MESH AND ISTIO



WHY MICROSERVICES CREATE NETWORKING COMPLEXITY

- Many services talk to each other.
- Communication patterns change frequently.
- Different protocols, ports, and versions.
- Hard to secure service-to-service traffic.
- Observability is scattered and inconsistent.
- Failures and latency are harder to diagnose.

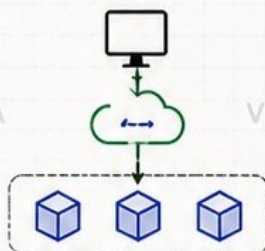


More services = more connections = more complexity.

EAST-WEST VS NORTH-SOUTH TRAFFIC

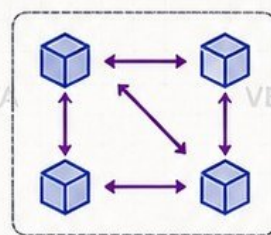
NORTH-SOUTH TRAFFIC

Traffic between external clients and the mesh.



EAST-WEST TRAFFIC

Traffic between services inside the mesh.



WHAT A SERVICE MESH SOLVES



TRAFFIC MANAGEMENT

- Intelligent routing
- Traffic splitting
- Canary & blue/green deployments
- Retries, timeouts, circuit breaking



SECURITY

- mTLS encryption
- Identity-based access control
- Authorization policies
- Secure service-to-service communication



OBSERVABILITY

- Metrics collection
- Distributed tracing
- Centralized logging
- Service dependency visibility



RELIABILITY

- Retries and fallback
- Circuit breaking
- Fault injection
- Prevent cascading failures

WHERE ISTIO FITS INTO KUBERNETES

CONTROL PLANE
(Istiod)
Configuration
Discovery
Certificates

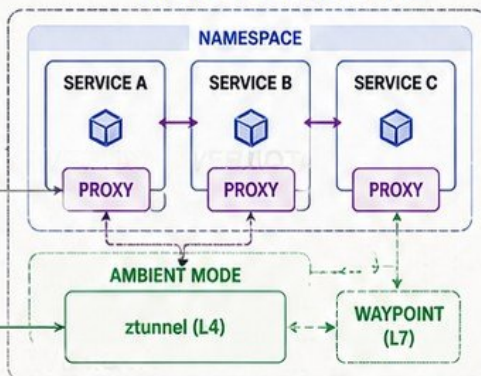


KUBERNETES CLUSTER



DATA PLANE

(Envoy / ztunnel)
Proxies handle traffic,
security, and telemetry.



→ **SIDE CAR MODE** (Envoy Proxy)

- - - → **AMBIENT MODE** (ztunnel + waypoints)



WHEN ISTIO IS USEFUL

- You run microservices at scale.
- You need advanced traffic control.
- You require mTLS and zero-trust security.
- You need deep observability and tracing.
- You operate in multi-team / multi-tenant environments.
- You need reliability and failure protection.



WHEN ISTIO MAY BE UNNECESSARY

- You run a monolith or very few services.
- Simpler networking needs that Kubernetes already handles.
- You have strict performance requirements with no sidecar/mesh overhead.
- Your team lacks operational readiness for the added complexity.
- You do not need advanced traffic, security, or observability.



Istio brings control, security, and visibility to service-to-service communication so you can build, run, and scale microservices with confidence.

VERIQTA – BUILD. SECURE. OBSERVE. SCALE.



youtube.com/@veriqta



github.com/veriqta



linkedin.com/company/veriqta



x.com/veriqta



instagram.com/veriqta



t.me/veriqta

2. ISTIO ARCHITECTURE

ISTIO CONTROL PLANE AND DATA PLANE

CONTROL PLANE (istiod)

Manages and configures the behavior of the mesh.



- Service discovery
- Configuration distribution
- Certificate management
- Policy enforcement
- Telemetry configuration

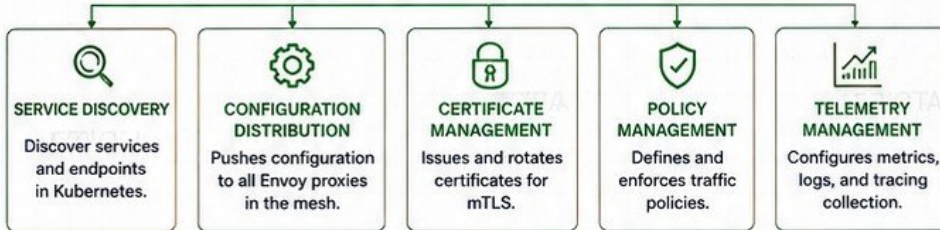
DATA PLANE (Envoy Proxy)

Handles all service-to-service traffic inside the mesh.

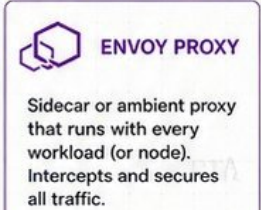


- L4 / L7 traffic processing
- Traffic interception
- Policy enforcement
- Telemetry collection
- mTLS encryption

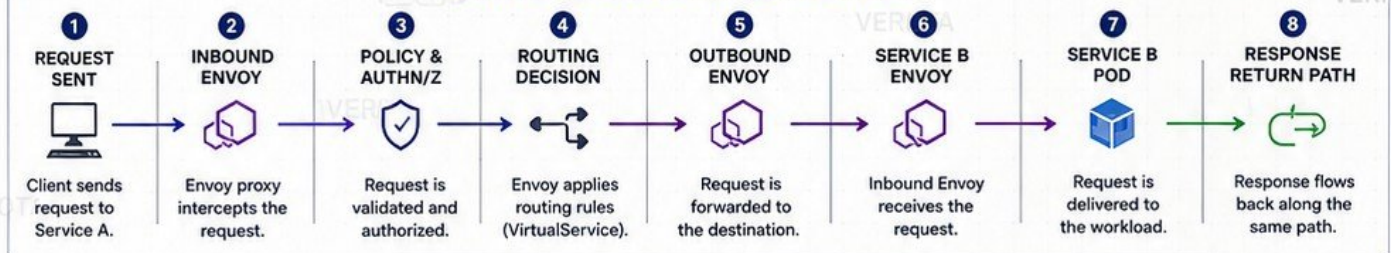
CONTROL PLANE COMPONENTS (istiod)



DATA PLANE COMPONENT



REQUEST LIFECYCLE THROUGH THE MESH



CONTROL-PLANE FAILURE VS DATA-PLANE FAILURE

CONTROL-PLANE FAILURE (istiod)

- No new configuration is pushed.
- New workloads may not get updated config.
- Existing workloads continue to run with last known good config.
- Data plane (Envoy proxies) keep forwarding traffic.
- Traffic continues, but changes cannot be made.

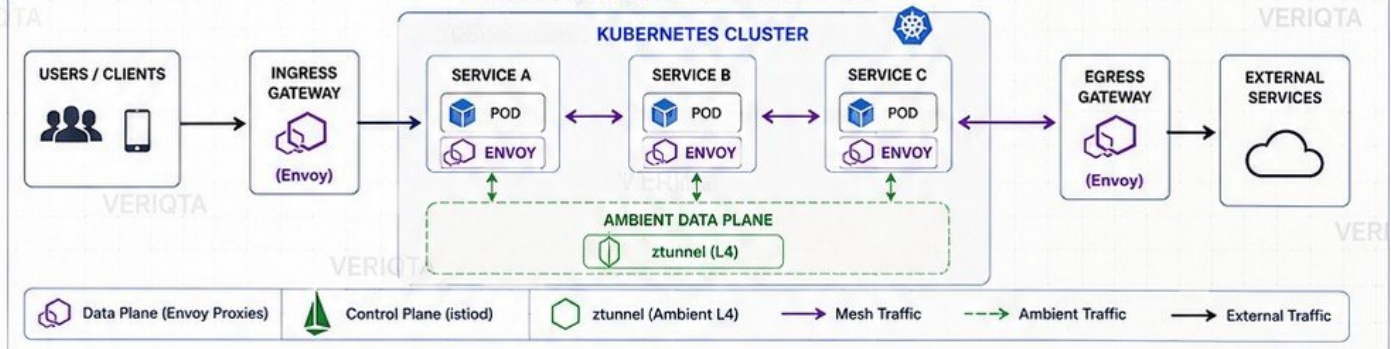


DATA-PLANE FAILURE (Envoy Proxy)

- Affected workload cannot send or receive traffic through the mesh.
- Requests may fail or timeout.
- Other workloads and proxies continue to function.
- Impact is isolated to the failed proxy/workload.



PRODUCTION ARCHITECTURE DIAGRAM

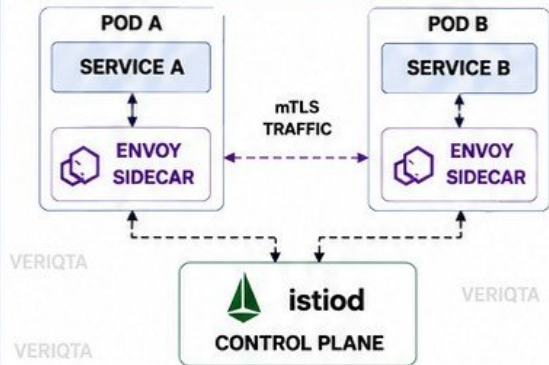


Istio separates concerns: istiod manages the mesh, Envoy proxies secure and route the traffic. This separation delivers scalability, reliability, and production-grade resilience.

VERIQTA SOCIAL

3. SIDECAR MODE AND AMBIENT MODE

TRADITIONAL ENVOY SIDECAR ARCHITECTURE



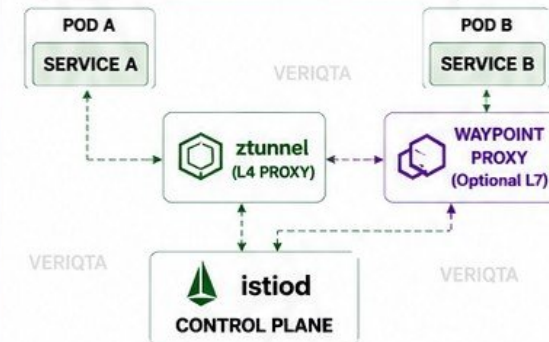
- Each pod gets an Envoy sidecar proxy.
- Envoy handles all inbound and outbound traffic.
- Sidecars communicate securely using mTLS.
- Full L7 capabilities in every pod.

SIDECAR INJECTION

- 1 Deploy application (without sidecar)
- 2 Admission webhook intercepts the pod creation
- 3 Envoy sidecar is injected automatically
- 4 Pod joins the mesh and starts securely communicating

★ Injection can be enabled per namespace or per workload using labels.

AMBIENT MESH ARCHITECTURE



- No sidecar in every pod.
- All traffic flows through ztunnel running on each node.
- ztunnel provides L4 capabilities (TCP, mTLS, traffic visibility).
- Waypoint proxies provide L7 (HTTP, policies, routing) when needed.
- Lower resource footprint.

ZTUNNEL (L4 DATA PLANE)

- ✓ Runs on every node.
- ✓ Handles all L4 traffic.
- ✓ mTLS encryption.
- ✓ Traffic visibility.
- ✓ No application changes.

WAYPOINT PROXIES (L7 DATA PLANE)

- ✓ Deployed in namespaces.
- ✓ Handle L7 processing when required.
- ✓ Routing, retries, auth, header-based policies, etc.
- ✓ Only for workloads that need L7.

L4 VERSUS L7 PROCESSING

L4 PROCESSING (ZTUNNEL)

- Operates at TCP layer.
- Very high performance.
- mTLS, traffic routing (by IP/port).
- No awareness of HTTP content.
- Lower resource usage.

L7 PROCESSING (WAYPOINT / SIDECAR)

- Operates at HTTP layer.
- Rich routing and traffic policies.
- Header, path, method awareness.
- Retries, timeouts, circuit breaking.
- Higher resource usage.

SIDECAR AND AMBIENT COEXISTENCE

SIDECAR MODE



AMBIENT MODE



COEXIST IN SAME CLUSTER



OPERATIONAL DIFFERENCES

	SIDECAR MODE	AMBIENT MODE
	Sidecar injected in every pod.	No sidecar in pods.
	More CPU & memory per pod.	Lower per-pod resource usage.
	L7 features available by default.	L7 features via waypoint proxies.
	Mature, widely adopted.	Newer, simpler operations.
	Troubleshoot per-pod proxies.	Troubleshoot node-level ztunnel.
	More moving parts per workload.	Fewer moving parts.

RESOURCE CONSIDERATIONS (TYPICAL)

SIDECAR MODE

- +50m to 200m CPU per pod (average)
- +64MB to 256MB memory per pod
- More proxies = more resource consumption

AMBIENT MODE

- ztunnel per node: ~100m to 250m CPU
- ~100MB to 300MB memory per node
- Better density and scalability

CHOOSING A DATA-PLANE MODEL

CHOOSE AMBIENT MODE IF:

- ✓ You want simpler operations.
- ✓ You want lower resource usage.
- ✓ Most of your traffic is L4.
- ✓ You are starting a new environment.
- ✓ You want to reduce sidecar overhead.

CHOOSE WHAT
FITS YOUR NEEDS.

CHOOSE SIDECAR MODE IF:

- ✓ You need advanced L7 features for most workloads.
- ✓ You rely heavily on HTTP routing and policies.
- ✓ You need very granular per-workload control.
- ✓ You already have production patterns with sidecars.
- ✓ You need mature features today.

BOTH MODES ARE POWERFUL. ISTIO GIVES YOU THE FLEXIBILITY TO START SIMPLE AND EVOLVE WITHOUT REWRITING YOUR APPLICATIONS.

4. INSTALLING ISTIO ON KUBERNETES



CLUSTER PREREQUISITES

- Kubernetes 1.24+
- kubectl configured
- Cluster admin access
- Outbound internet access
- Sufficient CPU and memory



ISTIOCTL

- Istio command line tool
- Download latest version
- Add istioctl to your PATH
- Verify installation



ISTIO PROFILES

- Default: balanced for most use cases
- Demo: for evaluation and demos
- Minimal: smallest footprint
- Custom profiles available



INSTALLING THE CONTROL PLANE

- Install Istio control plane (istiod)
- Creates istio-system namespace
- Adds CRDs, RBAC, and core components
- Configure with the desired profile



VERIFYING INSTALLATION

- Check control plane pods
- Validate Istio components
- Run analysis for best practices
- Ensure everything is healthy



NAMESPACE CONFIGURATION

- Use istio-system for control plane
- Create application namespaces
- Configure trust domain if needed
- Plan namespace strategy



ENABLING SIDE CAR INJECTION

- Label namespace for injection
- Automatic Envoy sidecar injection
- Verify injection on deployment
- Check sidecar resources



ENABLING AMBIENT MODE

- Install CNI plugin (ztunnel)
- Enable ambient profile
- Namespace enrollment for ambient
- Verify ztunnel participation



CHECKING MESH COMPONENTS

- Control plane pods (istiod)
- Data plane (Envoy or ztunnel)
- Gateways and add-ons
- Confirm all components running



BASIC INSTALLATION TROUBLESHOOTING

- Pods not starting
- Image pull errors
- RBAC permission issues
- Network/DNS connectivity
- Use logs and analyze for help

ESSENTIAL COMMANDS

1 istioctl version



CHECK ISTIOCTL
AND ISTIO VERSION

2 istioctl install



INSTALL ISTIO
CONTROL PLANE

3 kubectl get pods -n istio-system



VERIFY CONTROL PLANE
PODS ARE RUNNING

4 istioctl analyze



ANALYZE CONFIGURATION
AND BEST PRACTICES



TIP: ALWAYS RUN "istioctl analyze" AFTER INSTALLATION AND AFTER CHANGES
TO CATCH MISCONFIGURATIONS EARLY.

VERIQTA SOCIAL

5. BRINGING APPLICATIONS INTO THE MESH

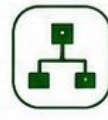


1 DEPLOYING A SAMPLE MICROSERVICES APPLICATION

- Deploy a 3-tier application (frontend, service, backend)
- Each component runs in its own Pod
- Kubernetes Services expose the Pods
- Example using istio-bookinfo application

```
# Deploy Bookinfo sample application
kubectl apply -f https://raw.githubusercontent.com/istio/istio/release-1.22/samples/bookinfo/platform/kube/bookinfo.yaml
```

```
# Verify pods
kubectl get pods -n default
```



2 KUBERNETES SERVICES

- Services provide stable endpoints
- ClusterIP for internal communication
- Istio will manage traffic between these services
- Each service is automatically discoverable in the mesh

```
# List services
kubectl get svc -n default
# Example output
```

NAME	TYPE	CLUSTER-IP
productpage	ClusterIP	10.96.123.10
reviews	ClusterIP	10.96.203.11
ratings	ClusterIP	10.96.55.12
details	ClusterIP	10.96.87.13



3 WORKLOAD IDENTITIES

- Every workload gets a unique identity
- Based on Kubernetes Service Account (KSA)
- Used for mTLS and authorization
- Identity format: <namespace>/<service-account>

```
# Check service accounts
kubectl get sa -n default
# Example: default service account
default
```



4 NAMESPACE ENROLLMENT

- Enable Istio in the namespace
- Sidecar injection or ambient mode
- This allows resources in the namespace to join the mesh

```
# Label namespace for sidecar injection
kubectl label namespace default istio-injection=enabled
# Verify label
kubectl get ns default --show-labels
```



5 AUTOMATIC SIDECAR INJECTION

- New Pods get Envoy sidecar automatically
- No application code changes
- Envoy handles traffic to/from the Pod
- Transparent to the application

```
# Redeploy a workload to trigger injection
kubectl rollout restart deployment/productpage -n default
# Check pods
kubectl get pods -n default -o wide
```



6 AMBIENT ENROLLMENT

- Enable ambient mode in namespace
- ztunnel handles L4 traffic
- Waypoint proxies handle L7 (when needed)
- Applications run without sidecars

```
# Enable ambient profile (example)
istioctl install --set profile=ambient
# Label namespace for ambient
istioctl x namespace label default ambient
```



7 VERIFYING ENVOY SIDECARS

- Check that Envoy sidecar is injected
- Sidecar container name: istio-proxy
- Status should be Running

```
# List pods with sidecars
kubectl get pods -n default -o wide
# Check containers in pod
kubectl describe pod <pod-name> -n default
```



8 VERIFYING ZTUNNEL PARTICIPATION

- For ambient mode, pods connect to ztunnel
- ztunnel manages L4 traffic securely
- Verify ztunnel pods and connections

```
# Check ztunnel pods
kubectl get pods -n istio-system -l app=ztunnel
# Check ztunnel status
istioctl ztunnel-status
```



9 TESTING SERVICE-TO-SERVICE COMMUNICATION

- All internal traffic goes through the mesh
- Traffic is automatically secured and observed
- Test connectivity between services

```
# Exec into a pod
kubectl exec -it deployment/productpage -n default -- /bin/bash
# Test internal service
curl -s http://reviews.default.svc.cluster.local:9080/reviews/0
```



10 UNDERSTANDING TRAFFIC BEFORE AND AFTER MESH ENROLLMENT

- Before: Plain Kubernetes networking
- After: Encrypted, observed, and managed by Istio
- Benefits: Security, traffic control, visibility

BEFORE MESH

- No mTLS
- No traffic visibility
- No traffic control
- No retries / timeouts
- No circuit breaking

AFTER MESH

- mTLS enabled
- Full visibility (metrics, logs, traces)
- Traffic routing & policies
- Retries, timeouts, circuit breaking
- Fault injection & testing



TIP:

Once applications are in the mesh, you can start applying traffic management, security policies, and observability features.

VERIQTA SOCIAL

6. VIRTUALSERVICE AND INTELLIGENT TRAFFIC ROUTING

VIRTUALSERVICE FUNDAMENTALS

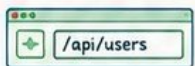
- VirtualService defines traffic routing rules to services inside the mesh.
- It lets you control how requests are matched and where they are routed.
- It does NOT define the destination subset (that's DestinationRule).
- It is namespace-scoped.

KEY CONCEPTS

- 1 **Hosts** - The destinations this rule applies to.
- 2 **HTTP Routes** - Define how HTTP requests are matched and routed.
- 3 **Match Conditions** - Decide which requests match the route.
- 4 **Route Destinations** - Where matched requests go.
- 5 **Traffic Percentages** - Split traffic across destinations.
- 6 **Service Versions** - Route traffic to specific versions using subsets.

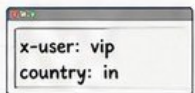
MATCH CONDITIONS

URI MATCHING



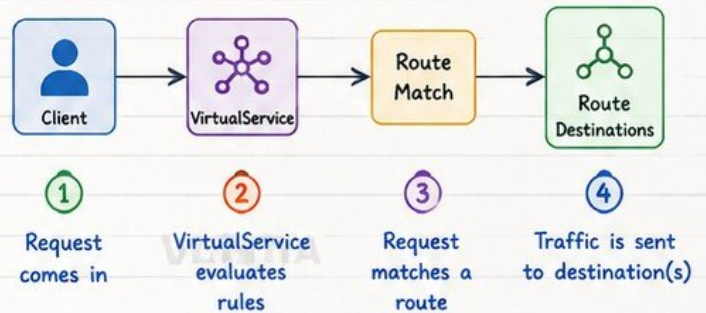
- exact: /api/users
- prefix: /api
- regex: ^/api/v[0-9]+/users

HEADER MATCHING

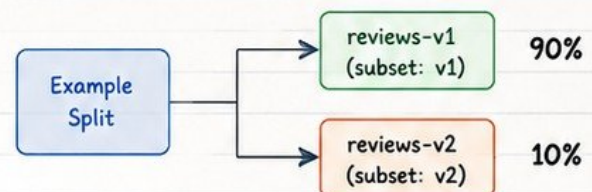


- exact match
- prefix match
- regex match

ROUTING FLOW



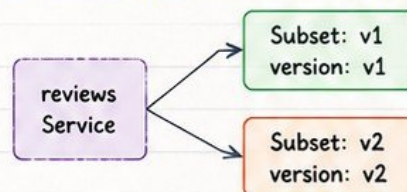
ROUTE DESTINATIONS & TRAFFIC PERCENTAGES



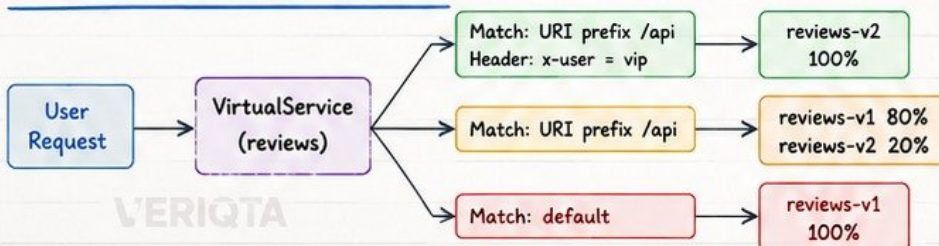
★ Percentages must add up to 100 for a given route. Istio uses weighted round-robin.

SERVICE VERSIONS USING SUBSETS

- VirtualService sends traffic to subsets defined in DestinationRule.
- Each subset represents a version of your service.



PRODUCTION ROUTING EXAMPLE



This setup:

- ✓ Routes VIP users to v2
- ✓ Sends regular traffic with 80/20 split
- ✓ All other traffic to v1
- ✓ Fully controlled inside mesh

7. DESTINATIONRULE, SUBSETS, AND LOAD BALANCING

DESTINATIONRULE OVERVIEW

- DestinationRule defines traffic policies applied to a service after a request is routed to it.
- It does NOT perform routing. VirtualService does that.
- It tells Istio HOW to communicate with the selected destination.
- It is namespace-scoped.

DEFINING SERVICE SUBSETS

- Subsets group versions or any set of pods using labels.
- Common use: version-based subsets (v1, v2, etc.).

Example

```
apiVersion: networking.istio.io/v1beta1
kind: DestinationRule
metadata:
  name: reviews-dr
spec:
  host: reviews.default.svc.cluster.local
  subsets:
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
```

- ★ Subsets are referenced by VirtualService route destinations.

TLS SETTINGS

- Configure how Istio handles TLS for this service.
- Modes: DISABLE, SIMPL, MUTUAL.

Example

```
trafficPolicy:
  tls:
    mode: ISTIO_MUTUAL
```

VIRTUALSERVICE AND DESTINATIONRULE RELATIONSHIP



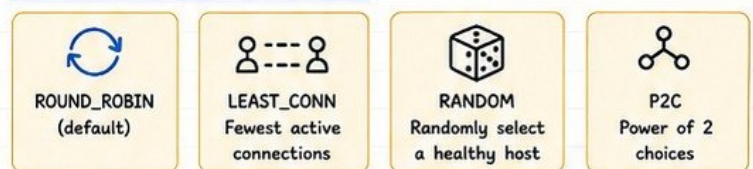
DESTINATIONRULE COMPONENTS



TRAFFIC POLICIES (WHAT CAN BE CONFIGURED)

- Load-balancing strategies
- Connection pools
- TLS settings
- Outlier detection
- Locality load balancing
- Custom protocols

LOAD-BALANCING STRATEGIES



★ Configured under trafficPolicy.loadBalancer.simple

CONNECTION POOLS

Example

```
trafficPolicy:
  connectionPool:
    tcp:
      maxConnections: 100
      connectTimeout: 2s
    http:
      http1MaxPendingRequests: 50
      maxRequestsPerConnection: 10
      idleTimeout: 30s
```

- ★ Controls how many connections and requests are opened to upstream hosts. Prevent resource exhaustion and improve performance.

OUTLIER DETECTION

Example

```
trafficPolicy:
  outlierDetection:
    consecutive5xxErrors: 5
    interval: 10s
    baseEjectionTime: 30s
    maxEjectionPercent: 50
```

- ★ Automatically ejects unhealthy hosts so traffic is not sent to bad instances.

COMMON CONFIGURATION MISTAKES

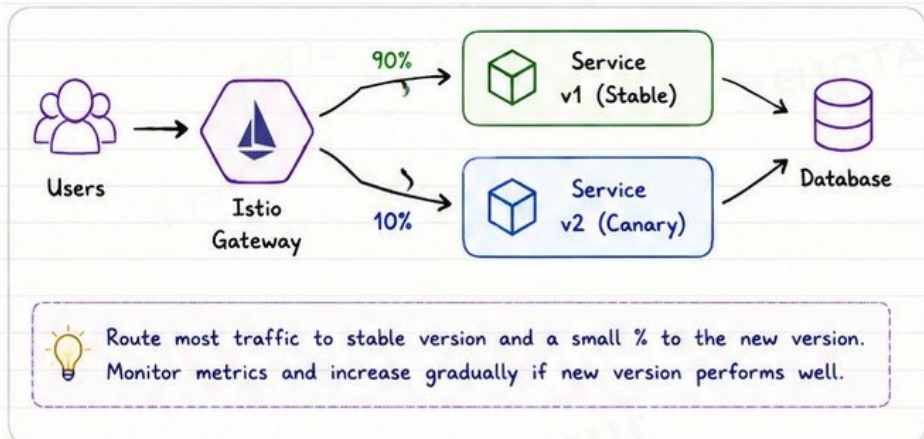
❌ SUBSET LABEL MISMATCH	Labels in DestinationRule do not match actual pod labels. Result: no endpoints.
❌ FORGETTING DESTINATIONRULE	VirtualService routes to subsets, but no DestinationRule exists. Result: default policies only.
❌ WRONG HOST	DestinationRule host does not match the service FQDN. Result: policies not applied.
❌ POLICY OVERRIDES NOT UNDERSTOOD	Multiple DestinationRules for same host can cause unexpected behavior.
❌ INVALID TRAFFIC POLICY	Incorrect fields or unsupported values. Result: config ignored or errors.

8. CANARY, BLUE-GREEN, AND PROGRESSIVE DELIVERY

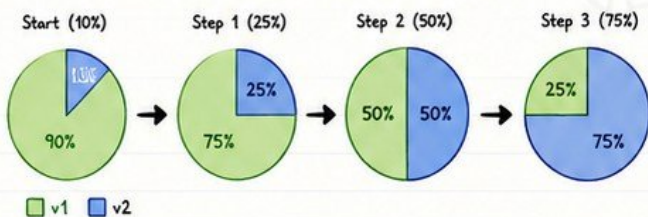
KEY CONCEPTS

- Canary deployment architecture
- 90/10 traffic splitting
- Gradually increasing traffic
- Blue-green deployments
- Header-based routing
- User-based routing
- A/B testing
- Testing new versions without changing application code
- Rollback strategy
- Production deployment workflow

CANARY DEPLOYMENT ARCHITECTURE



GRADUALLY INCREASING TRAFFIC



★ Increase traffic gradually while observing metrics, logs, and user feedback.

BLUE-GREEN DEPLOYMENTS



Deploy new version to Green → Test → Switch traffic → Monitor → Keep or Rollback to Blue

ROUTING TECHNIQUES

HEADER-BASED ROUTING

Route requests based on HTTP headers.

Example:

If header "x-version" = "beta"
→ send to v2
Else → send to v1

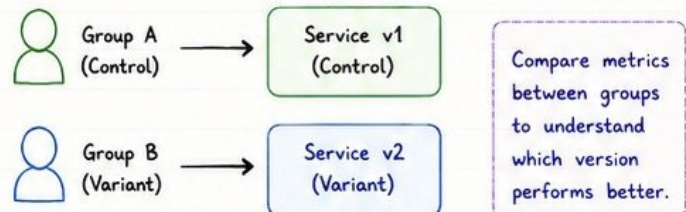
USER-BASED ROUTING

Route based on user identity or consistent hashing.

Example:

Users in group "beta-users"
→ send to v2
Others → send to v1

A/B TESTING



ROLLBACK STRATEGY

- ✓ Monitor key metrics (error rate, latency, traffic).
- ✓ If issues detected → stop traffic ramp-up.
- ✓ Shift 100% traffic back to stable version.
- ✓ Investigate and fix issues.
- ✓ Redeploy new version when ready.



PRODUCTION DEPLOYMENT WORKFLOW



★ Always monitor, validate and be ready to rollback.

EXAMPLE:
90/10 CANARY
(VirtualService)

```
http:
  - route:
    - destination:
        host: myapp
        weight: 90
    - destination:
        host: myapp
        subset: v2
        weight: 10
```

This routes 90% of traffic to v1 and 10% to v2.

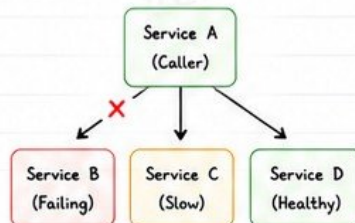
9. RESILIENCE: RETRIES, TIMEOUTS, AND CIRCUIT BREAKING



Failures are normal in distributed systems. Resilience patterns help services recover quickly and prevent small issues from becoming outages.

1. DISTRIBUTED-SYSTEM FAILURES

- Network latency and packet loss
- Service overload
- Instance crashes or restarts
- Database slowness
- Temporary dependency outages
- Partial failures (some instances failing)



2. REQUEST TIMEOUTS



- Set a deadline for requests.
- Fail fast instead of waiting indefinitely.
- Protects threads, connections and resources.
- Timeouts should be slightly longer than the 99th percentile latency.

Too short → False failures
Too long → Slow failure detection

3. RETRY POLICIES



- Retries improve success rate for transient failures.
- Use limited retries with exponential backoff.
- Do not retry non-idempotent operations by default.

4. RETRY BUDGETS AND RISKS



- Too many retries increase load.
- Can amplify outages.
- Use retry budgets (e.g., max 20%) of total traffic.
- Monitor and adjust based on error rate.

More retries ≠ More reliability
Smart retries = Better resilience

5. CONNECTION POOLS



- Reuse connections to reduce latency.
- Limit max connections.
- Configure idle timeout and max pending requests.
- Prevents resource exhaustion.

Example (DestinationRule):
maxConnections: 100
http1MaxPendingRequests: 50
maxRequestsPerConnection: 10

6. CIRCUIT-BREAKING CONCEPTS

- Stop sending requests to a failing service.
- Allow time for recovery.
- Three states:



Prevents constant retries to a failing service and protects the entire system.

7. OUTLIER DETECTION

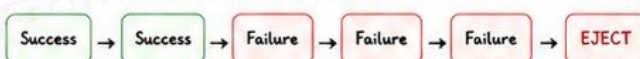


- Automatically eject unhealthy instances from load balancing.
- Based on consecutive failures, error rate, or latency.
- Ejected instances are retried after a cool-down period.

Example (OutlierDetection):
consecutive5xx: 5
interval: 10s
baseEjectionTime: 30s

8. CONSECUTIVE FAILURES

Track failures to detect unhealthy instances.



Use thresholds to avoid reacting to random failures.

9. PREVENTING CASCADING FAILURES



- Use timeouts, retries, and circuit breakers together.
- Limit concurrency and connection pools.
- Apply bulkheads (isolate resources per service).
- Fail fast and degrade gracefully.
- Observe, alert, and act early.

10. DESIGNING RESILIENT SERVICE COMMUNICATION



★ Resilience is not a single feature. It is the combination of patterns working together.

10. FAULT INJECTION AND FAILURE TESTING



Deliberately introducing failures helps you validate resilience, verify retry and timeout behavior, and ensure your applications can survive real production incidents.

1. WHY INTRODUCE FAILURES?

- Find weaknesses before users do.
- Verify timeouts, retries, and circuit breaking.
- Improve system reliability and recovery.
- Build confidence in new deployments.



Fail Fast.
Learn Faster.
Stay Reliable.

2. FAULT INJECTION TYPES

DELAY INJECTION

Adds latency before a request is sent to the destination.



ABORT INJECTION

Terminates the request with a specific HTTP error code.

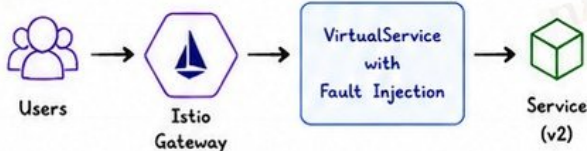


HTTP ERROR SIMULATION

Returns a specific HTTP status code without contacting the backend.



3. HOW FAULT INJECTION WORKS (EXAMPLE FLOW)



VirtualService controls traffic and applies faults before it reaches the destination service.

4. EXAMPLE: VIRTUALSERVICE WITH FAULT INJECTION

```

apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
    - reviews
  http:
    - route:
        - destination:
            host: reviews
            subset: v2
            weight: 100
      fault:
        delay:
          percentage:
            value: 50
            fixedDelay: 2s
        abort:
          percentage:
            value: 10
            httpStatus: 503
  
```

50% of requests delayed by 2s

10% of requests aborted with HTTP 503

40% of requests succeed normally

5. TESTING BEHAVIORS

Scenario	What to Test	Expected Behavior
Timeout Behavior	Inject delay longer than request timeout.	Requests timeout and fallback/retry triggers.
Retry Behavior	Inject errors (e.g., 503) and enable retries.	Requests retried based on retry policy.
Resilience Validation	Inject random faults during load testing.	Service remains available and stable.
Circuit Breaking Validation	Inject consecutive failures.	Circuit opens and traffic is stopped temporarily.

6. COMBINING ROUTING AND FAULT INJECTION



★ Test the new version with faults while most users stay on the stable version.

8. PRODUCTION READINESS TESTING



Validate timeouts, retries, and circuit breaking.



Simulate failures under load.



Measure error rates and latency.



Verify graceful degradation.



Document recovery and rollback steps.

7. SAFE TESTING PRACTICES

- Test in non-production environments first.
- Limit fault percentage (start small).
- Use time-boxed tests.
- Monitor metrics, logs, and traces.
- Communicate with your team.
- Never inject faults blindly in production.
- Clean up test configurations after testing.
- Automate rollback if error rate is high.



9. BENEFITS OF FAULT INJECTING

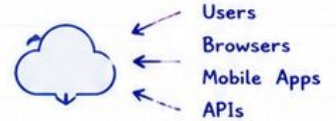
- Builds confidence in deployments
- Reveals hidden weaknesses
- Improves user experience during failures
- Prevents cascading failures
- Ensures production-ready applications



11. ISTIO GATEWAYS AND INGRESS TRAFFIC

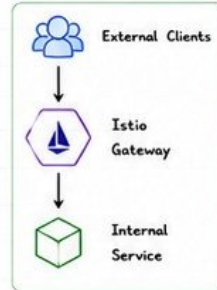


Gateways handle NORTH-SOUTH traffic coming from outside the cluster and securely route it to services inside the mesh.



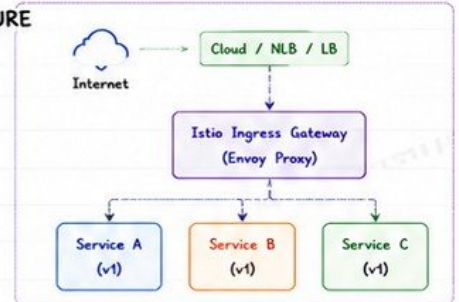
1. NORTH-SOUTH TRAFFIC

- Traffic from external clients to services inside the cluster.
- Enters through an Istio Gateway.
- Routes to internal services via VirtualService.
- Different from east-west traffic (service to service inside mesh).



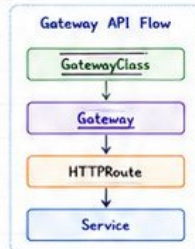
2. GATEWAY ARCHITECTURE

- Load balancer exposes the Ingress Gateway.
- Istio Gateway listens on ports and hosts.
- VirtualService routes requests to services.
- mTLS secures traffic inside the mesh.



3. KUBERNETES GATEWAY API

- Modern API for managing networking across clusters.
- Standard resources:
 - GatewayClass
 - Gateway
 - HTTPRoute
- More portable and expressive than Ingress.



4. INGRESS GATEWAY

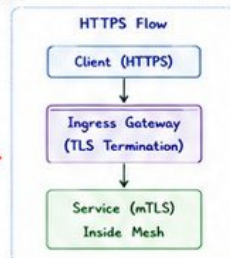
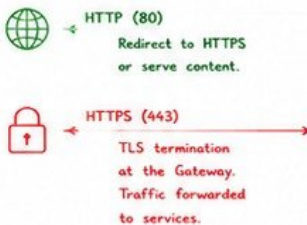
- Usually deployed in istio-system namespace.
- Works with a Load Balancer (cloud or on-prem).
- Handles HTTP, HTTPS, and TLS traffic.
- Scales independently for high availability.



Common Ports

80 → HTTP
443 → HTTPS
15021 → Health Check
15443 → TLS

5. HTTP AND HTTPS TRAFFIC



6. HOST CONFIGURATION

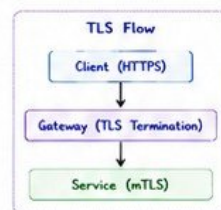
- Gateways use Hosts to match incoming requests.
- Each host can have different routing rules.
- Supports wildcards (e.g., *.example.com).

```
Gateway Example
apiVersion: networking.istio.io/v1beta1
kind: Gateway
metadata:
  name: app-gateway
spec:
  selector:
    istio: ingressgateway
  servers:
    - port:
        number: 443
        name: https
        protocol: HTTPS
      hosts:
        - app.example.com
        - api.example.com
```

Multiple hosts can share the same gateway.

7. TLS TERMINATION

- TLS is terminated at the Gateway.
- Certificates can be managed by k8s secrets.
- After termination, traffic is routed using HTTP headers.
- mTLS is used from Gateway to services.



8. ROUTING EXTERNAL REQUESTS

- VirtualService defines routing rules.
- Routes based on host, path, headers, etc.
- Can route to different versions or services.
- Supports rewrites and redirects.

VirtualService Example

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: app-vs
spec:
  hosts:
    - app.example.com
  gateways:
    - app-gateway
  http:
    - match:
        - uri:
            prefix: /route-to-600
          destination:
            host: app-service
            subset: v1
```

9. GATEWAY-TO-SERVICE TRAFFIC

- After routing, traffic goes from Gateway to Service.
- Traffic inside mesh uses mTLS automatically.
- Retries, timeouts, and policies apply as usual.



Best Practice

- Use dedicated Ingress Gateway for external traffic.
- Enable HSTS and secure TLS ciphers.
- Monitor Gateway metrics and access logs.
- Keep Gateway and VirtualService configs simple and clear.
- Test all hosts and paths before production.

10. TROUBLESHOOTING FAILED INGRESS

Problem	Check	Commands / Actions
No response / Timeout	LB, Gateway pods, Security Groups	kubectl get svc -n istio-system kubectl get pods -n istio-system
404 Not Found	Host / Path mismatch	Check Gateway & VirtualService hosts
503 Service Unavailable	Backend service issues	kubectl get endpoints <service> istioctl proxy-status
TLS Errors	Certificate / Secret issues	kubectl get secret <secret> openssl s_client -connect host:443
Traffic not routed correctly	VirtualService rules	istioctl analyze, istioctl proxy-config routes <pod>

12 Egress Traffic and External Services

★ Traffic leaving the mesh

- When a workload needs to call a service outside the mesh, traffic leaves through the data plane.
- By default, traffic may go directly to the external service.

★ ServiceEntry

- ServiceEntry tells Istio about external services.
- Allows the mesh to apply policies, routing, and observability to outbound traffic.
- Without ServiceEntry, Istio has no visibility or control.

★ Registering external services

- Define host, ports, protocol, and location.
- Supports DNS names or external IPs.
- Makes external services first-class citizens in the mesh.

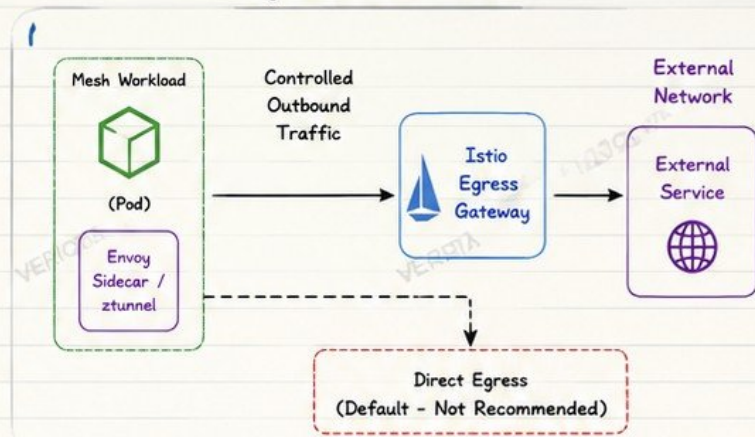
★ Controlling outbound communication

- Apply traffic policies, retries, timeouts, and TLS settings.
- Monitor metrics and traces for external calls.
- Enforce security and compliance controls.

★ Egress gateways

- Route outbound traffic through a dedicated gateway.
- Central point for policy enforcement, auditing, and control.
- Useful for IP allowlisting, logging, and compliance.

Egress Traffic Flow



ServiceEntry Example

```

apiVersion: networking.istio.io/v1beta1
kind: ServiceEntry
metadata:
  name: external-api
spec:
  hosts:
    - api.paymentprovider.com
  location: MESH_EXTERNAL
  ports:
    - number: 443
      name: https
      protocol: HTTPS
  resolution: DNS
  
```

Key Points

- ✓ Defines external service inside the mesh
- ✓ Enables policy, routing, metrics, and tracing
- ✓ DNS resolution for external host
- ✓ Secure HTTPS communication

External HTTPS Services

- Use HTTPS for encrypted communication.
- Configure TLS settings in DestinationRule.
- Validate certificates to prevent MITM.
- Support for SNI and custom CA if needed.



DNS Considerations

- Istio relies on DNS for external resolution.
- Use stable DNS names, not changing IPs.
- Control DNS at the cluster or node level.
- Consider caching and TTL impacts.



Restricting Uncontrolled Egress

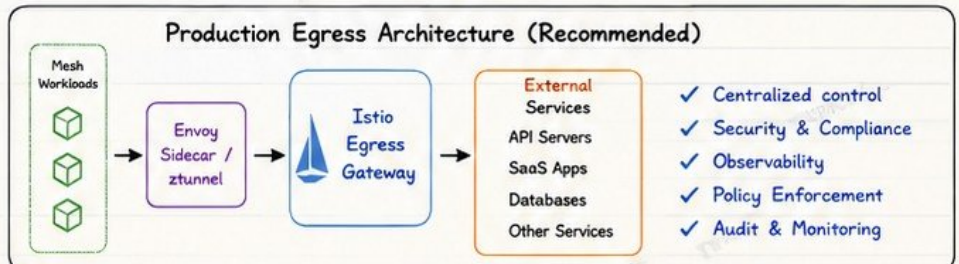
- Block direct access to internet.
- Allow only through egress gateway.
- Use Kubernetes NetworkPolicies.
- Apply Istio DENY by default policies.



Security Implications

- Uncontrolled egress can lead to data leakage.
- Attackers can exfiltrate data via external calls.
- Use egress gateway for centralized visibility.
- Apply least privilege for outbound access.

Production Egress Architecture (Recommended)



Always register external services with ServiceEntry and route traffic through an egress gateway for security, visibility, and control.

13 Mutual TLS and Workload Identity

★ Zero-trust service communication

Every service verifies the identity of the service it communicates with.

★ Workload Identity

- Each workload has a unique identity in the mesh.
- Identity is bound to a Kubernetes ServiceAccount.

★ Service Accounts

- Kubernetes ServiceAccount is the foundation of workload identity.
- Istio maps ServiceAccount to a workload identity.

★ Istio Certificates

- Istio automatically provisions X.509 certificates for each workload.
- Certificates are short-lived and rotated automatically.

★ Automatic Certificate Rotation

- Certificates are rotated before expiry without downtime.
- Rotation is handled by istiod and Envoy.

★ Mutual TLS (mTLS)

- Both client and server present and verify certificates.
- Ensures encrypted and authenticated communication between workloads.

Verifying Encrypted Service Traffic

Check connection details (from a pod)

```
curl -v http://service-b.default.svc.cluster.local
```

Check certificate information

```
openssl s_client -connect service-b.default.svc:80
```

Verify mTLS status in Envoy

```
istioctl proxy-config secret <pod> -o json
```

Check PeerAuthentication status

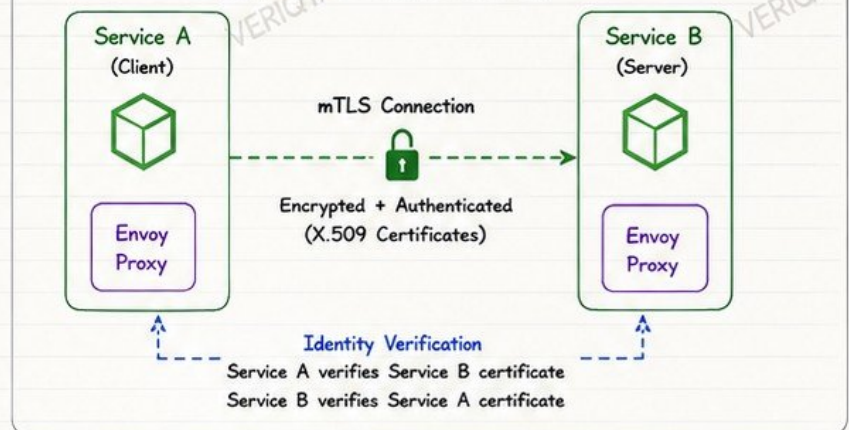
```
kubectl get peerauthentication -A
```



Key Benefits

- Encrypts all service-to-service traffic.
- Prevents eavesdropping and man-in-the-middle attacks.
- Provides strong identity-based authentication.
- Supports zero-trust security model.

How mTLS Works in Istio

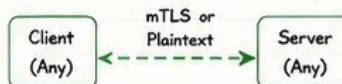


PeerAuthentication

Controls the mTLS behavior for workloads in a namespace.

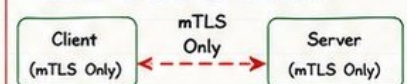
PERMISSIVE (Default)

- Accepts both mTLS and plaintext traffic.
- Does not require clients to use mTLS.
- Use for gradual adoption.



STRICT

- Accepts only mTLS traffic.
- Requires all clients to present valid certificates.
- Use for full zero-trust enforcement.



Migrating Safely to STRICT mTLS

- 1 Inventory workloads and dependencies.
- 2 Ensure all workloads have sidecar injection enabled.
- 3 Run in PERMISSIVE mode and monitor traffic.
- 4 Fix workloads that cannot communicate (certificates, policies).
- 5 Enable STRICT mode in a test namespace first.
- 6 Monitor metrics, logs, and traces.
- 7 Roll out STRICT mode to more namespaces gradually.



mTLS Production Checklist

- | | |
|---------------------------------------|---------------------------------|
| ✓ Sidecar injection enabled | ✓ Metrics and alerts in place |
| ✓ PeerAuthentication configured | ✓ Certificate rotation working |
| ✓ istiod healthy | ✓ Policies aligned with mTLS |
| ✓ No plaintext traffic in STRICT mode | ✓ Traffic verified as encrypted |



mTLS is the foundation of zero-trust in a service mesh. Start PERMISSIVE, observe, then move to STRICT when ready.

14

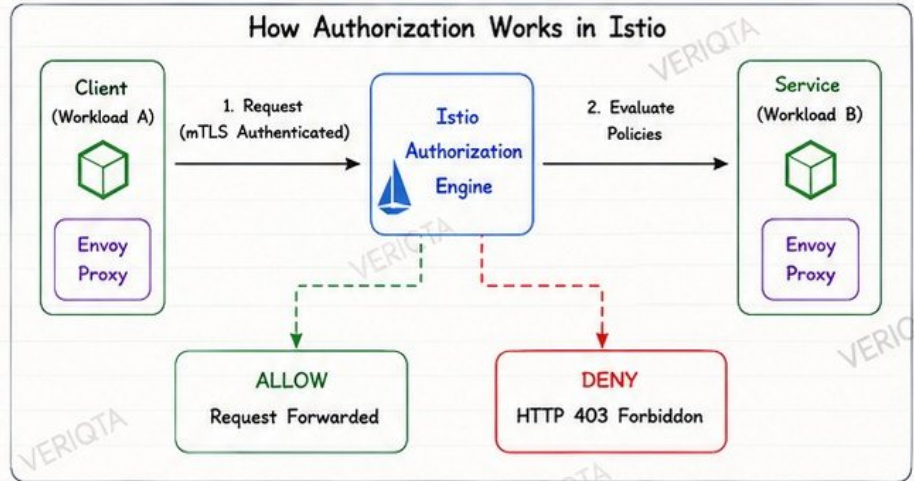
AuthorizationPolicy and Zero-Trust Access

★ Authentication vs Authorization

- Authentication verifies who a workload is (identity).
- Authorization determines what an authenticated workload is allowed to do (access).
- Istio Authentication (mTLS, JWT) happens before Authorization.

★ AuthorizationPolicy

- AuthorizationPolicy defines allow or deny rules for workloads.
- Policies are evaluated by Istio authorization engine on each request.
- Policies are scoped to namespace and workloads.
- If no ALLOW policy matches, the request is denied (deny-by-default).



ALLOW Policies

- Explicitly allow specific traffic.
- Example: Allow only Service A to call Service B.
- If no ALLOW policy matches, access is denied.

```

apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-from-a-to-b
  namespace: default
spec:
  selector:
    matchLabels:
      app: service-b
  rules:
    - from:
        - source:
            principals: ["cluster.local/ns/default/sa/service-a"]
      to:
        - operation:
            methods: ["GET", "POST"]
            paths: ["/api/*"]
  
```

DENY Policies

- Explicitly deny unwanted traffic.
- Useful to block specific sources, paths, or methods.
- Deny policies take precedence over allow.

```

apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: deny-admin-path
  namespace: default
spec:
  selector:
    matchLabels:
      app: service-b
  action: DENY
  rules:
    - to:
        - operation:
            paths: ["/admin/*"]
      from:
        - source:
            notPrincipals: ["cluster.local/ns/trusted/*"]
  
```

Service-to-Service Authorization

- Control which services can communicate.
- Combine source identity and destination rules.
- Protect internal APIs between microservices.

Namespace-Level Policies

- Apply policies to all workloads in a namespace.
- Use for baseline security and deny-by-default.
- Example: Block all traffic except from allowed namespaces.

Workload-Level Policies

- Target specific workloads using labels.
- More granular control for sensitive services.
- Useful for tiered or multi-tenant applications.

Service-Account-Based Authorization

- Use principals based on service accounts.
- Best practice for zero-trust identity control.
- Works with mTLS workload identity.

Deny-by-Default Architecture

- Start by denying all traffic.
- Add explicit allow rules as needed.
- Enforces least privilege by design.

🔧 Troubleshooting Unexpected 403 Responses

- 1 Check if the request is mTLS authenticated.
- 2 Verify AuthorizationPolicies in the namespace and workload.
- 3 Look for DENY policies blocking the request.
- 4 Check for missing ALLOW policies (deny-by-default).
- 5 Validate source principals and service accounts.
- 6 Review policy scope: namespace vs workload selector.
- 7 Inspect Istio logs and Envoy access logs.
- 8 Use: `istioctl authz check <pod> --from <source> --to <destination>`

⚠️ Common 403 Causes

- ✓ No matching ALLOW policy (deny-by-default).
- ✓ DENY policy explicitly blocking the request.
- ✓ Wrong source principal or service account.
- ✓ Policy applied in unexpected namespace.
- ✓ Path, method, or host not allowed.
- ✓ mTLS not established (authentication failed).
- ✓ PeerAuthentication in STRICT but client not in mesh.

🛡️ Least-Privilege Principles

- ✓ Grant only what is necessary.
- ✓ Prefer ALLOW with precise rules.
- ✓ Avoid wildcard sources.
- ✓ Use service accounts for identity.
- ✓ Continuously review and refine policies.
- ✓ Monitor and audit authorization decisions.



AuthorizationPolicy is the foundation of zero-trust in Istio. Authenticate first, then authorize only the traffic that should be allowed.

15

JWT Authentication and API Security

★ RequestAuthentication

- Validates JWTs in incoming requests.
- Extracts identity from the token.
- Does NOT allow or deny access that is done by AuthorizationPolicy.
- Supports multiple issuers and JWKS.

★ JSON Web Tokens (JWT)

- Compact, URL-safe claims format.
- Contains signed claims about the user.
- Commonly used for API authentication.
- Three parts: Header . Payload . Signature.

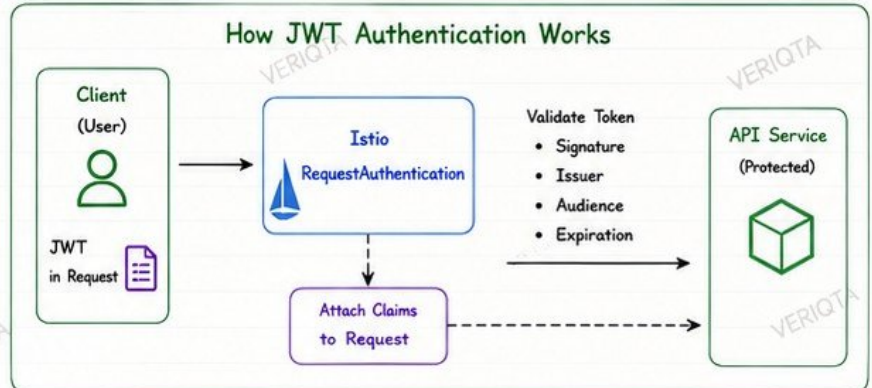
JWT Structure

Header Base64Url . Payload Base64Url . Signature Base64Url

★ JWT Validation in Istio

- RequestAuthentication verifies:
 - Token signature
 - Token issuer (iss)
 - Audience (aud)
 - Expiration (exp)
- If valid, claims are attached to the request as headers for authorization.

How JWT Authentication Works



Issuers (iss)

- The authority that issued the token.
- Eg: `https://auth.example.com`
- Must match exactly.

JWKS (JSON Web Key Set)

- Public keys used to verify the token signature.
- Istio fetches keys from the JWKS endpoint.
- Keys rotate automatically.

Claims

- Information about the user.
- Common claims:
 - sub - user id
 - email - user email
 - roles - roles/permissions
 - exp - expiration time
 - aud - intended audience

Combining RequestAuthentication with AuthorizationPolicy



Protecting API Endpoints

- Require valid JWT for all requests.
- Verify issuer and audience.
- Use AuthorizationPolicy to enforce fine-grained access.
- Support role-based or scope-based authorization.
- Secure both internal and external APIs.

RequestAuthentication Example

```

apiVersion: security.istio.io/v1
kind: RequestAuthentication
metadata:
  name: jwt-auth
  namespace: default
spec:
  selector:
    matchLabels:
      app: api
  jwtRules:
    - issuer: "https://auth.example.com"
      jwksUri: "https://auth.example.com/.well-known/jwks.json"
      audiences:
        - "my-api"
      forwardOriginalToken: true
  
```

AuthorizationPolicy Using Claims

```

apiVersion: security.istio.io/v1
kind: AuthorizationPolicy
metadata:
  name: allow-admins
  namespace: default
spec:
  selector:
    matchLabels:
      app: api
  rules:
    - from:
        - source:
            requestPrincipals: ["*"]
      when:
        - key: request.auth.claims[roles]
          values: ["admin", "superuser"]
      action: ALLOW
  
```

User Identity vs Workload Identity

User Identity (JWT)	Workload Identity (mTLS / SPIFFE)
✓ Represents a human user. ✓ Comes from an identity provider (IdP). ✓ Used to secure APIs and user actions. ✓ Claims describe the user (roles, email, scopes).	✓ Represents a workload (service or pod). ✓ Based on Istio certificates and SPIFFE IDs. ✓ Used for service-to-service authentication. ✓ No user claims.



Common Authentication Failures

Invalid Signature

Token signature does not match JWKS key.
Fix: Check JWKS URL and key rotation.

Invalid Issuer

Token issuer does not match configuration.
Fix: Verify issuer (iss) value.

Expired Token

Token has expired (exp time is in the past).
Fix: Re-authenticate and get new token.

Invalid Audience

Audience (aud) claim does not match.
Fix: Ensure audience is correct.

Missing or Invalid Claims

Required claims are missing or have wrong values.
Fix: Check claim names and values.

Clock Skew

Time difference between systems causes failure.
Fix: Sync system time (NTP).



JWT + AuthorizationPolicy = Strong API Security. Authenticate the user, then authorize the action. Verify, then trust (only what is allowed).

16

Istio Observability

★ Mesh Telemetry

- Istio automatically collects rich telemetry from all services.
- Provides deep visibility into traffic behavior and performance.
- Helps you understand, monitor, and troubleshoot your applications.

★ Key Observability Signals

**Metrics**

Quantitative measurements of traffic and system behavior.

**Logs**

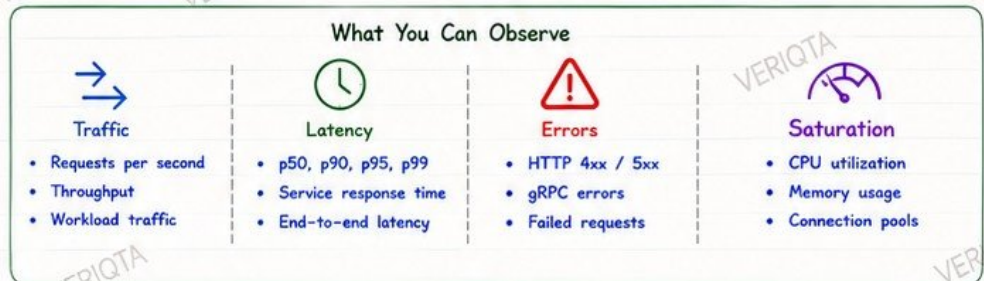
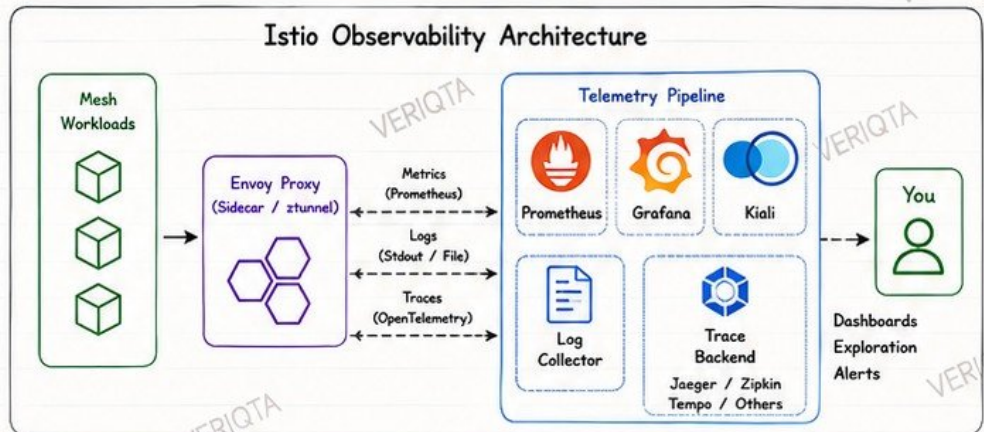
Detailed request and proxy logs for investigation.

**Distributed Tracing**

Request flow across multiple services and systems.

★ Golden Signals in Istio

- ✓ **Latency** - How long requests take
- ✓ **Traffic** - How much and how many
- ✓ **Errors** - Failed requests and codes
- ✓ **Saturation** - Resource utilization

**Prometheus**

- Default metrics backend for Istio.
- Scrapes metrics from Envoy and Istio components.
- Stores time-series data.

Port: 9090

**Grafana**

- Visualization and dashboarding.
- Pre-built Istio dashboards.
- Create custom dashboards for services and infrastructure.

Port: 3000

**Kiali**

- Service mesh observability UI.
- Topology graph
- Traffic flow
- Metrics, logs, traces
- Health and configuration

Port: 20001

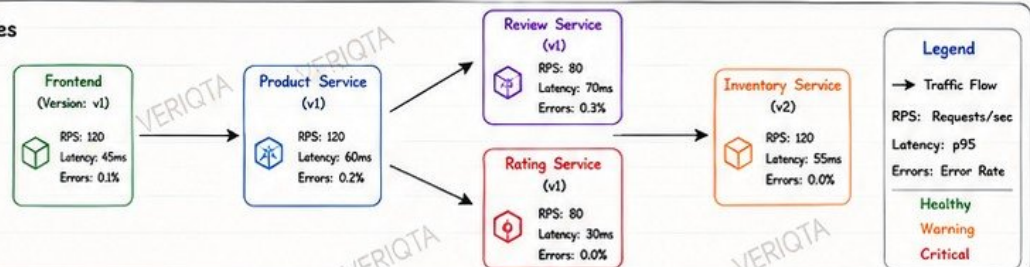
**Trace Backends**

- Stores distributed traces.
- Common options:
 - Jaeger
 - Zipkin
 - Tempo

Varies (e.g., 16686)

Understanding Service Dependencies

- Kiali Service Graph shows real-time communication between services.
- Visualize request volume, latency, and errors for each service-to-service edge.
- Helps identify bottlenecks, failures, and high-error services.



Example Observability Dashboards



Mesh Dashboard - Overview of the entire mesh.
Workload Dashboard - Metrics per service.
Service Dashboard - Deep dive into a service.
Performance Dashboard - Latency and saturation.
Error Dashboard - Error rates and failures.

**Best Practices**

- ✓ Ensure all telemetry is collected and stored.
- ✓ Use meaningful labels for services and versions.
- ✓ Set up alerts for errors and latency.
- ✓ Use tracing to debug complex request flows.
- ✓ Review dashboards regularly.

**Sample Prometheus Queries**

Request rate:
`sum(rate(istio_requests_total[5m])) by (destination_service)`
Error rate (5xx):
`sum(rate(istio_requests_total{response_code=~"5.."}[5m]))`
p95 latency:
`histogram_quantile(0.95, sum(rate(istio_request_duration_milliseconds_bucket[5m])) by (le, destination_service))`



Observability is the key to reliability. With metrics, logs, and traces, you can understand what is happening and fix issues before your users are affected.

17

Troubleshooting Istio in Production

★ Why Troubleshooting Matters

- Complex meshes can fail in many ways.
- Problems may be in the app, mesh config, network, or infrastructure.
- Use observability and `istioctl` to isolate and fix issues quickly.

Troubleshooting Workflow

- 1. Observe Symptoms**
Identify failing request or component.
- 2. Collect Data**
Use `istioctl`, metrics, logs, and traces.
- 3. Narrow Down**
Check config, traffic path, policies.
- 4. Fix and Validate**
Apply fix and verify traffic & metrics.



Common Production Issues



Sidecar Injection Failures

- Pods missing Envoy sidecar.
- Check namespace label and injector status.



mTLS Mismatches

- Connection failures due to TLS issues.
- Verify PeerAuthentication and certs.



Routing Failures

- Wrong route, weight, host, or header match.
- Check VirtualService and DestinationRule.



Authorization Failures

- 403 errors from AuthorizationPolicy.
- Check policy scope and service accounts.



Gateway Failures

- Ingress not working or TLS issues.
- Check Gateway, VirtualService, and certs.



Upstream / Service Failures

- No healthy endpoints or app errors.
- Check endpoints, health checks, and logs.



Essential `istioctl` Commands

<code>istioctl analyze</code>	Analyze mesh configuration
<code>istioctl proxy-status</code>	Check Envoy proxy health
<code>istioctl proxy-config clusters <pod></code>	Inspect clusters (upstream services)
<code>istioctl proxy-config listeners <pod></code>	Inspect listeners (ports & protocols)
<code>istioctl proxy-config routes <pod></code>	Inspect routes (L7 routing rules)
<code>istioctl proxy-config endpoints <pod></code>	Inspect endpoints discovered by Envoy



Quick Tip

Always start with: `istioctl analyze` and `istioctl proxy-status`
They quickly reveal many common issues.



Deep Dive with `istioctl`



Clusters

Shows upstream services.
Check if the service exists, is healthy, and endpoints are available.



Listeners

Shows open ports and protocols.
Useful for checking inbound traffic configuration.



Routes

Shows HTTP routes, hosts, paths, and route destinations.
Helps debug 404, 503, routing loops, or wrong matches.



Endpoints

Shows endpoints discovered by Envoy for each cluster.
Check for missing or unhealthy endpoints.



Envoy Configuration Inspection

Envoy is the data plane.
Most issues can be diagnosed by inspecting Envoy's live config.

What You Can Inspect

- ✓ Clusters (upstream services)
- ✓ Listeners (ports & protocols)
- ✓ Routes (L7 rules)
- ✓ Endpoints (service instances)
- ✓ Secrets (certificates)
- ✓ Bootstrap (proxy metadata)



Pro Tip

Use `-o json` for full output and `jq` for powerful filtering.

```
istioctl proxy-config routes <pod> -o json | jq '.dynamic_route_configs[].route_config'
```

Troubleshooting Playbook (Step-by-Step)

- 1 Check Mesh Health**
`istioctl analyze`
`istioctl proxy-status`
- 2 Verify Traffic Flow**
Check metrics, logs, and traces for the failing request.
- 3 Inspect Envoy Config**
clusters, listeners, routes, endpoints using `istioctl`.
- 4 Check Policies**
mTLS, AuthorizationPolicy, RequestAuthentication, PeerAuthentication.
- 5 Validate External Access**
Gateways, DNS, certificates, and network paths.
- 6 Fix and Verify**
Apply fix, monitor metrics, logs, and traces.



```
istioctl get pods -n istio-system # Check control plane
istioctl proxy-status             # Check all Envoy proxies
istioctl analyze -A               # Analyze all namespaces
```

```
istioctl proxy-config clusters <pod> -o wide # Clusters with details
istioctl proxy-config routes <pod> --name http.8080 # Routes on specific listener
istioctl pc endpoints <pod> --cluster outbound[80] | service.namespace.svc.cluster.local # Endpoints for a service
```



Istio provides powerful tools to see what is happening inside the mesh.
Observe, inspect, understand, then fix. This is the key to stable production meshes.

18 Production Incident Scenarios

★ Each scenario follows the same structure: Symptoms → Investigation → Commands → Config Checks → Root Cause → Resolution

1	Scenario 1: Service A suddenly cannot communicate with Service B.	
Symptoms	Service A errors, timeouts, or connection refused when calling B.	
Investigation	Check connectivity, endpoints, routing, mTLS, and policies.	
Commands	<code>istioctl proxy-status</code> <code>istioctl proxy-config clusters <pod-a></code> <code>istioctl proxy-config endpoints <pod-a></code>	
Config Checks	- VirtualService / DestinationRule - ServiceEntry (if external) - PeerAuthentication (mTLS) - AuthorizationPolicy	
Root Cause	Service B endpoints missing or unhealthy, policy blocking, mTLS mismatch, or bad routing.	
Resolution	Fix endpoints/policy/routing or mTLS settings. Verify traffic flow after changes.	

2	Scenario 2: Application returns 503 after enabling Istio.	
Symptoms	503 errors from app after sidecar injection or mesh enablement.	
Investigation	Check sidecar injection, readiness, health checks, and resource limits.	
Commands	<code>kubectl get pods -o wide</code> <code>istioctl proxy-status</code> <code>kubectl describe pod <pod></code>	
Config Checks	- Sidecar injection label - Readiness/Liveness probes - Resource limits (CPU/Memory) - DestinationRule timeouts	
Root Cause	Sidecar not injected, probe failures, or Envoy resources causing restarts.	
Resolution	Enable injection, fix probes/resources, redeploy and verify health.	

3	Scenario 3: A new release receives 100% of traffic instead of 10%.	
Symptoms	All traffic going to new version. Old version receives none.	
Investigation	Check VirtualService weights, subset labels, and route match.	
Commands	<code>istioctl proxy-config routes <pod></code> <code>kubectl get vs -o yaml</code> <code>kubectl get dr -o yaml</code>	
Config Checks	- VirtualService weights - Subset labels in DestinationRule - Route match conditions - Host / URI mismatches	
Root Cause	Wrong weights, missing subset labels, or unmatched route rule.	
Resolution	Fix weights/subsets, ensure labels match, and reload configuration.	

4	Scenario 4: Requests fail after STRICT mTLS is enabled.	
Symptoms	503 UF, 495, or connection failures between services.	
Investigation	Check PeerAuthentication, certs, and SAN mismatch.	
Commands	<code>istioctl authn tls-check <pod></code> <code>istioctl proxy-config secret <pod></code> <code>istioctl proxy-config clusters <pod></code>	
Config Checks	- PeerAuthentication (STRICT) - DestinationRule TLS settings - SAN / Service hostname - ServiceAccount mismatch	
Root Cause	Workload not using mTLS, cert mismatch, or missing service identity.	
Resolution	Fix identities, rotate certs if needed, ensure mTLS is configured correctly.	

5	Scenario 5: AuthorizationPolicy blocks legitimate requests.	
Symptoms	403 Forbidden errors in services or specific endpoints.	
Investigation	Review policies, principals, and match conditions.	
Commands	<code>istioctl x authz check <pod> --from <source></code> <code>kubectl get authorizationpolicy -A -o yaml</code> <code>istioctl proxy-config routes <pod></code>	
Config Checks	- AuthorizationPolicy rules - From / To / When conditions - Namespace selectors - ServiceAccount names	
Root Cause	Policy denies traffic due to incorrect match, missing principal, or default deny.	
Resolution	Update policy rules, add missing principals, or adjust scope as needed.	

6	Scenario 6: Ingress gateway works internally but not externally.	
Symptoms	Services reachable inside cluster but not from outside.	
Investigation	Check Gateway, DNS, LB, Security Groups, and certs.	
Commands	<code>kubectl get svc -n istio-system</code> <code>dig +short myapp.example.com</code> <code>istioctl proxy-config listeners <gateway-pod></code>	
Config Checks	- Gateway / VirtualService hosts - Service type (LoadBalancer) - DNS / TTL - Cloud NSG / Security Group rules	
Root Cause	DNS misconfig, LB not exposed, wrong host, or firewall blocking traffic.	
Resolution	Fix DNS/LB rules, update hosts/certs, and verify external access.	



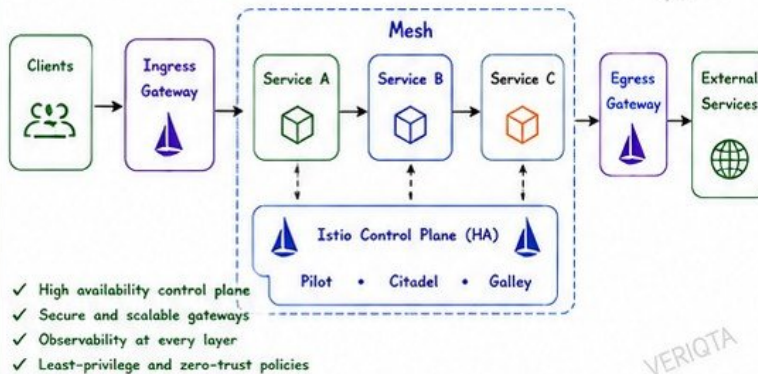
Always observe first, then investigate methodically. Use `istioctl`, logs, metrics, and traces to identify the real issue before making changes.

19

Production Architecture and Operational Best Practices

★ A reliable, observable, secure, and maintainable service mesh is built by design, not by default.

Designing Istio for Production



Control-plane Availability

- Deploy control plane with at least 3 replicas per component.
- Spread across multiple nodes and zones.
- Use PodDisruptionBudgets (PDBs) to protect availability.
- Monitor control plane health continuously.

Component	Recommended Replicas
istiod (Pilot)	3
istiod (Citadel)	3
istio-galley	3
istio-ingressgateway	2+
istio-egressgateway	2+



Resource Requests and Limits

- Set appropriate CPU and memory requests and limits for all Istio components and proxies.
- Prevent noisy neighbor problems.
- Review and tune regularly.



Proxy Resource Consumption

- Envoy sidecars consume CPU/memory.
- Monitor proxy metrics (CPU, RSS, requests, connections).
- Optimize filters, logging, and tracing.
- Consider ambient mode for scale.



Namespace Strategy

- Use dedicated namespaces for system, gateways, and teams.
- Avoid placing all workloads in a single namespace.
- Use workload and namespace scoping for policies.



Configuration Ownership

- Define clear ownership for Gateways, VirtualServices, Policies, and Security.
- Use GitOps for configuration and change management.
- Maintain documentation and change history.



Policy Management

- Start simple. Add policies incrementally.
- Use AuthorizationPolicy with deny-by-default where possible.
- Regularly review and audit policies and exceptions.



Certificate Considerations

- Use cert-manager or CSI driver for certificate lifecycle.
- Enable automatic rotation and short-lived certificates.
- Secure private keys and limit certificate access.



Gateway Scaling

- Scale ingress/egress gateways based on traffic and connections.
- Use HPA for horizontal scaling.
- Tune Envoy concurrency and connection limits.
- Use multiple gateways for isolation if needed.



Observability Requirements

- Collect metrics (Prometheus) from proxies and control plane.
- Use Kiali for topology and insights.
- Centralize logs (ELK / Loki).
- Enable distributed tracing (Jaeger / Tempo).



Upgrade Planning

- ✓ Test upgrades is staging first.
- ✓ Follow upgrade path (minor → patch).
- ✓ Review breaking changes and notes.
- ✓ Backup Istio configuration.
- ✓ Validate after upgrade.



Sidecar vs Ambient Operational Decisions

- Sidecar mode: more features, more resource overhead.
- Ambient mode: lighter, simpler, better for scale.
- Choose per use case and maturity.



Configuration Validation

- Use istioctl analyze to validate configurations.
- Use CI/CD checks for policies, Gateways, and VirtualServices.
- Prevent invalid or conflicting configurations from reaching prod.



Avoid Overly Complex Mesh Policies

- Keep policies readable and maintainable.
- Avoid chaining too many rules.
- Group by intent, not by implementation detail.
- Simplicity improves reliability.



- ✓ Control plane is highly available
- ✓ All components have resource limits
- ✓ Gateways are secured and scaled
- ✓ Observability is fully configured
- ✓ mTLS is enabled for service traffic

Production Checklist

- ✓ Policies follow least privilege
- ✓ Certificates rotate automatically
- ✓ Backups and disaster recovery defined
- ✓ Upgrade path and rollback plan ready
- ✓ Configuration is validated before deploy

- ✓ Monitoring and alerts are in place
- ✓ Logs and traces are centralized
- ✓ Runbooks and troubleshooting docs ready
- ✓ Team training and ownership defined



A successful mesh is not just about enabling Istio. It's about operating it with resilience, visibility, security, and simplicity at scale.

20 Complete Production Lab and Reference



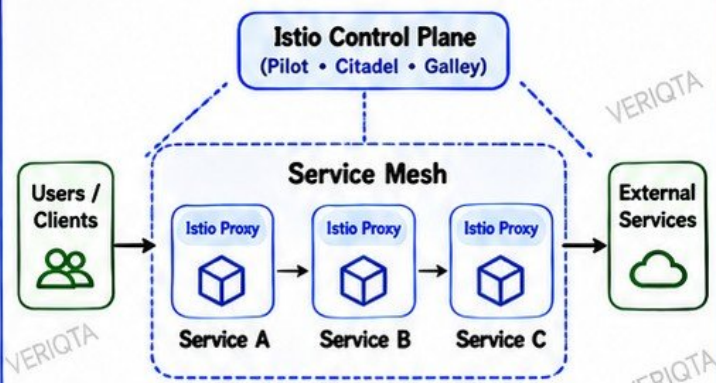
Build a complete Istio environment and validate production readiness.



Complete Istio Production Lab

- 1 Deploy multiple microservices.
- 2 Add workloads to the mesh.
- 3 Configure service versions.
- 4 Create traffic routing.
- 5 Perform a 90/10 canary deployment. 10%
- 6 Configure retries and timeouts.
- 7 Add circuit-breaking behavior.
- 8 Enable mTLS.
- 9 Apply service-to-service authorization.
- 10 Configure external ingress.
- 11 Control external service access.
- 12 Generate test traffic.
- 13 Inspect metrics and service relationships.
- 14 Introduce a controlled failure.
- 15 Troubleshoot the failure.
- 16 Roll back the deployment.

Istio Production Architecture



Production Flow



Key Commands



- `istioctl analyze`
- `istioctl proxy-status`
- `istioctl proxy-config clusters <pod>`
- `istioctl proxy-config listeners <pod>`
- `istioctl proxy-config routes <pod>`
- `istioctl proxy-config endpoints <pod>`



Final Production Checklist

- ☒ Traffic management verified
- ☒ mTLS verified
- ☒ Authorization policies tested
- ☒ Ingress tested
- ☒ Egress reviewed

- ☒ Metrics available
- ☒ Logs available
- ☒ Tracing tested
- ☒ Failure scenarios tested
- ☒ Recovery procedure documented